

SmartSim: Un Sistema de Aprendizaje Activo Guiado por Incertidumbre para Reducir Evaluaciones de Simuladores Costosos

R. Indira Valentina Requiz Molina¹

¹Proyecto independiente

Agosto de 2026

Abstract

Los simuladores de sistemas físicos de alta fidelidad son computacionalmente costosos, lo que limita la cantidad de evaluaciones que se pueden ejecutar al explorar un espacio de parámetros. Presentamos **SmartSim**, un sistema de aprendizaje activo que combina un modelo sustituto probabilístico (Proceso Gaussiano) con un simulador de alta fidelidad, en el cual una función de adquisición decide en cada iteración qué punto del espacio de parámetros conviene evaluar con el simulador real. Formalizamos el sistema, incluyendo un mecanismo de parada basado en la incertidumbre máxima observada sobre una región candidata. Sobre el benchmark sintético de Branin, y con un protocolo multi-semilla, el aprendizaje activo alcanza un error objetivo en 22.1 ± 2.7 evaluaciones reales (15 semillas), mientras que un muestreo aleatorio equivalente necesita una mediana de 762 evaluaciones (30 semillas, presupuesto extendido a 2000 para eliminar la censura de presupuesto; 3 de 30 semillas no convergen ni así) — una reducción real de aproximadamente 97%, muy por encima del 65% que un presupuesto fijo de 60 evaluaciones sugeriría de forma engañosa. El 94.7% de las predicciones del modelo caen dentro de un intervalo de $\mu \pm 2\sigma$. Además, evaluamos explícitamente el criterio de parada por incertidumbre (H2): funciona, pero solo cuando el umbral τ se calibra en la escala real del kernel entrenado, no en la escala intuitiva del error del problema — un hallazgo de diseño que reportamos junto con el trade-off medido entre evaluaciones ahorradas y calibración preservada. El código está disponible públicamente.

1 Introducción

1.1 El cuello de botella computacional

Caracterizar el comportamiento de un sistema físico o dinámico complejo bajo distintas condiciones iniciales o parámetros de diseño requiere, en general, ejecutar un simulador de alta fidelidad repetidas veces. Cuando cada ejecución cuesta segundos, minutos u horas de cómputo, el número de evaluaciones que se pueden pagar dentro de un presupuesto razonable es limitado, y ese costo escala mal con la dimensionalidad del espacio de parámetros.

Una solución estándar es reemplazar el simulador por un *modelo sustituto* (surrogate model) entrenado sobre un conjunto de evaluaciones previas, y usarlo como aproximación barata. El riesgo de este enfoque es evidente: un sustituto entrenado sobre una cantidad limitada de datos puede fallar silenciosamente en regiones del espacio que no fueron bien exploradas.

1.2 El problema

No proponemos minimizar únicamente el error de predicción del sustituto. El problema que nos interesa es distinto:

minimizar el número de evaluaciones del simulador de alta fidelidad, sujeto a un requisito de confiabilidad predictiva del modelo sustituto.

Esta formulación desplaza el centro de gravedad del proyecto: no se trata de qué tan bueno es el sustituto en promedio, sino de si el sistema *sabe cuándo no sabe*, y puede usar esa señal para decidir cuándo seguir consultando al simulador real.

1.3 Pregunta de investigación

¿Puede la incertidumbre predictiva de un modelo sustituto probabilístico usarse como criterio operacional para decidir cuándo una simulación de alta fidelidad sigue siendo necesaria?

1.4 Hipótesis

Hipótesis 1 (Central). *Un modelo sustituto probabilístico puede aprender activamente de un simulador de alta fidelidad y utilizar su propia incertidumbre para determinar cuándo debe volver a consultar al simulador, reduciendo significativamente el costo de simulación sin degradar la precisión ni la confiabilidad de sus predicciones.*

Esta hipótesis se descompone en dos partes, ambas evaluadas empíricamente en este trabajo (§5, §6) con protocolo multi-semilla:

- H1** Guiar el muestreo mediante una función de adquisición basada en incertidumbre reduce el número de evaluaciones del simulador necesarias para alcanzar un error objetivo, respecto a muestreo aleatorio, sin degradar la precisión final. **(evaluada: 15 semillas para aprendizaje activo, 30 para el baseline con presupuesto extendido a 2000 evaluaciones)**
- H2** Un criterio de parada basado en la incertidumbre máxima sobre una región candidata permite detener la consulta al simulador preservando la confiabilidad de las predicciones. **(evaluada: barrido de τ sobre 5 semillas por valor; el mecanismo funciona pero exige calibrar τ en la escala del kernel entrenado, no en la escala del error del problema)**

1.5 Contribuciones

1. Una arquitectura de aprendizaje activo *simulator-in-the-loop* que acopla un sustituto probabilístico con un simulador de alta fidelidad mediante un mecanismo de decisión consciente de la incertidumbre.
2. Una formalización matemática precisa del sistema tal como está implementado, incluyendo la distinción explícita entre la función de adquisición (qué punto muestrear) y el criterio de parada (cuándo dejar de simular).
3. Una implementación de código abierto y un protocolo experimental reproducible para evaluar la eficiencia de consultas al simulador, junto con un reporte honesto de qué resultados son preliminares y qué experimentos faltan.

2 Trabajo relacionado

2.1 Modelado sustituto y emulación con Procesos Gaussianos

El uso de Procesos Gaussianos como sustitutos de códigos computacionales costosos es una técnica establecida en ingeniería de confiabilidad y diseño de experimentos por computadora. Marrel e Iooss [1] ofrecen una revisión reciente centrada específicamente en la estimación y validación de la distribución predictiva de los GP, y advierten que un GP puede tener buena predictividad puntual y, al mismo tiempo, intervalos de predicción mal calibrados — una distinción central para este trabajo.

2.2 Aprendizaje activo y emulación adaptativa

Svendsen et al. [2] proponen un esquema de aprendizaje activo secuencial para construir emuladores de códigos costosos mediante la optimización de una función de adquisición, en un problema con estructura similar al nuestro. Moustapha, Marelli y Sudret [3] presentan un marco modular general para aprendizaje activo en confiabilidad estructural, descomponiendo el problema en modelo sustituto, función de aprendizaje y criterio de parada — la misma descomposición conceptual que adoptamos en §4, aunque aplicada a un problema de regresión en vez de estimación de probabilidad de falla.

2.3 Optimización bayesiana

Las funciones de adquisición que implementamos (UCB, EI, PI) provienen de la literatura clásica de optimización bayesiana [4, 5]. A diferencia del uso típico de estas técnicas — encontrar un óptimo global con el menor número de evaluaciones — nuestro interés principal no es la ubicación del óptimo sino la caracterización confiable de la superficie de respuesta completa, lo cual motiva incluir también una variante de exploración pura por varianza.

2.4 Frente a nuestro trabajo

No afirmamos que el uso de GP con aprendizaje activo para reducir evaluaciones de simuladores costosos sea, en sí mismo, novedoso: la literatura citada demuestra lo contrario. Nuestra contribución es acotada y específica: (a) una implementación que separa explícitamente “dónde muestrear” de “cuándo parar” como dos mecanismos independientes, y (b) un protocolo experimental honesto sobre qué parte de esa arquitectura está evaluada y cuál no.

3 Arquitectura del sistema

La Figura 1 resume el flujo del sistema. El simulador de alta fidelidad **nunca** es reemplazado por el sustituto: el sustituto solo decide, en cada iteración, en qué punto conviene invertir la próxima evaluación real, y si conviene seguir evaluando en absoluto.

4 Formalización matemática

Esta sección formaliza el sistema *tal como está implementado* (módulos `surrogate.py`, `acquisition.py`, `orchestrator.py`), evitando presentar mecanismos más sofisticados de los que el código efectivamente ejecuta.

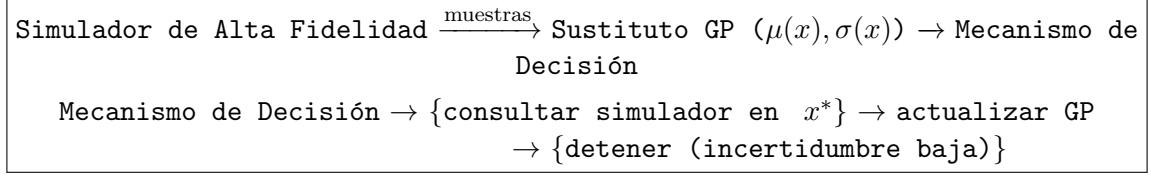


Figure 1: Arquitectura simulator-in-the-loop. El GP se reentrena en cada iteración activa sobre el historial acumulado.

4.1 Modelo sustituto

Sea $f : \mathcal{X} \rightarrow \mathbb{R}$ el simulador de alta fidelidad, con $\mathcal{X} \subset \mathbb{R}^d$ el espacio de parámetros. Dado un historial $D_t = \{(x_i, y_i)\}_{i=1}^{n_t}$ de evaluaciones reales acumuladas hasta la iteración t , el modelo sustituto es un Proceso Gaussiano

$$f(x) \mid D_t \sim \mathcal{GP}(\mu_t(x), k(x, x')) \quad (1)$$

con kernel compuesto

$$k(x, x') = \sigma_f^2 \cdot k_{\text{Matérn}}^{\nu=5/2}(x, x'; \ell) + \sigma_n^2 \delta(x, x') \quad (2)$$

donde $k_{\text{Matérn}}^{\nu=5/2}$ usa una longitud de escala independiente por dimensión $\ell \in \mathbb{R}^d$ (kernel anisotrópico ARD), y el término de ruido blanco captura ruido de observación. Antes de ajustar, $\mathcal{X}$ se normaliza a $[0, 1]^d$ mediante los límites del espacio de parámetros, y los targets y se estandarizan a media 0 y varianza 1. Los hiperparámetros del kernel se estiman por máxima verosimilitud marginal, con 5 reinicios del optimizador para mitigar mínimos locales.

Nota de implementación: el ajuste es completo en cada iteración activa — μ_t, σ_t se recalculan reentrenando el GP sobre todo D_t desde cero. No hay actualización incremental (rank-one update, sparse GP, etc.). Esto se declara como limitación en §7.

Para cualquier punto candidato x , el GP entrega media y desvío estándar predictivos:

$$\mu_t(x), \quad \sigma_t(x) = \sqrt{k_t(x, x)} \quad (3)$$

4.2 Función de adquisición: dónde muestrear

Sea y_t^* el mejor valor observado hasta la iteración t ($\min_i y_i$ o $\max_i y_i$ según el objetivo). Definimos $\Delta(x) = y_t^* - \mu_t(x) - \xi$ (minimización; signo invertido para maximización), con ξ un margen de exploración ($\xi = 0.01$ por defecto). El sistema implementa cuatro funciones de adquisición intercambiables:

$$a_{\text{UCB}}(x) = \pm \mu_t(x) + \kappa \sigma_t(x), \quad \kappa = 2.0 \quad (4)$$

$$a_{\text{EI}}(x) = \Delta(x) \Phi\left(\frac{\Delta(x)}{\sigma_t(x)}\right) + \sigma_t(x) \phi\left(\frac{\Delta(x)}{\sigma_t(x)}\right) \quad (5)$$

$$a_{\text{PI}}(x) = \Phi\left(\frac{\Delta(x)}{\sigma_t(x)}\right) \quad (6)$$

$$a_{\text{var}}(x) = \sigma_t(x) \quad (7)$$

donde Φ y ϕ son la CDF y PDF normales estándar. El próximo punto a evaluar es

$$x_t^* = \arg \max_{x \in C_t} a(x) \quad (8)$$

donde C_t es un conjunto de $|C_t| = 2000$ candidatos muestreados uniformemente en $\mathcal{X}$, generado de nuevo en cada iteración (optimización de la adquisición por búsqueda de Monte Carlo, no por gradiente). La variante a_{var} ignora y_t^* : no busca un óptimo, busca caracterizar exhaustivamente la superficie de respuesta.

4.3 Criterio de decisión: cuándo detener la consulta al simulador

Éste es el mecanismo que sostiene H2, y requiere precisión: **no** es una decisión punto a punto de “confiar en el sustituto vs. simular”. Es una decisión *global* sobre si toda la región candidata ya está suficientemente bien caracterizada.

Definimos la incertidumbre máxima observada entre los candidatos en la iteración t :

$$s_t = \max_{x \in C_t} \sigma_t(x) \quad (9)$$

y la regla de decisión, dado un umbral opcional τ :

$$D_t = \begin{cases} \text{DETENER,} & \text{si } \tau \text{ está definido y } s_t < \tau \\ \text{CONSULTAR SIMULADOR EN } x_t^*, & \text{en caso contrario} \end{cases} \quad (10)$$

Interpretación: el sistema no enruta predicciones individuales entre “barato” y “caro”. En cada iteración, o bien decide que *ninguna* región candidata justifica ya una nueva simulación (incertidumbre máxima bajo el umbral), o bien siempre consulta al simulador en el punto más prometededor según $a(\cdot)$. Es un criterio de parada epistatémica global, no un enrutador de confianza local por predicción.

Algorithm 1 Bucle de aprendizaje activo SmartSim

```

1:  $D_0 \leftarrow$  muestreo inicial por Latin Hypercube ( $n_{\text{init}}$  puntos, siempre evaluados en  $f$ )
2: for  $t = 1, 2, \dots, T_{\text{max}}$  do
3:   Reentrenar GP completo sobre  $D_{t-1} \rightarrow$  obtener  $\mu_t, \sigma_t$ 
4:    $C_t \leftarrow$  2000 candidatos uniformes en  $\mathcal{X}$ 
5:    $s_t \leftarrow \max_{x \in C_t} \sigma_t(x)$ 
6:   if  $\tau$  definido y  $s_t < \tau$  then
7:     detener
8:   end if
9:    $x_t^* \leftarrow \arg \max_{x \in C_t} a(x)$ 
10:   $y_t^* \leftarrow f(x_t^*)$  ▷ única llamada real al simulador en esta iteración
11:   $D_t \leftarrow D_{t-1} \cup \{(x_t^*, y_t^*)\}$ 
12: end for

```

4.4 Calibración de incertidumbre

Definimos la cobertura empírica a 2σ sobre un conjunto de prueba $\{(x_i, y_i)\}_{i=1}^N$ no usado para entrenar:

$$\text{Cobertura}_{2\sigma} = \frac{1}{N} \sum_{i=1}^N \mathbb{I} [|y_i - \mu_T(x_i)| \leq 2 \sigma_T(x_i)] \quad (11)$$

Bajo un GP bien calibrado y errores aproximadamente gaussianos, el valor nominal esperado es $\approx 95.4\%$. Esta métrica es, junto con H1, uno de los dos resultados empíricos reportados en §6.

5 Protocolo experimental

5.1 Baselines

- Muestreo aleatorio uniforme (baseline principal, implementado).
- Latin Hypercube Sampling de presupuesto fijo (pendiente).
- GP + adquisición aleatoria, GP + UCB, GP + PI (variantes de ablación, código disponible vía el parámetro `acquisition`, no corridas sistemáticamente todavía).

5.2 Benchmarks

- **Branin** (2D, sintético) — implementado y evaluado en este trabajo.
- **Hartmann-6** (6D, sintético) — implementado en el código (`simulator.py`), no evaluado aún en el protocolo de comparación.
- Un simulador dinámico simple (p.ej. masa-resorte-amortiguador) y un simulador realista de menor escala quedan como trabajo futuro (§8).

5.3 Protocolo de evaluación propuesto para trabajo futuro

Para que H1 y H2 queden evaluadas con el rigor que exige un reviewer, el protocolo mínimo es:

1. ≥ 20 –30 semillas aleatorias independientes por método (actualmente: 1 semilla para aprendizaje activo, 8 para el baseline aleatorio — comparación asimétrica, ver §6), **y un presupuesto ampliado para el baseline aleatorio hasta que la mayoría de sus semillas converjan realmente**, para eliminar la censura observada en §6.2.
2. Reporte de media, mediana, desvío estándar e intervalo de confianza del 95% para el número de evaluaciones necesarias.
3. Curva de error vs. número de evaluaciones reales, para cada método, con banda de dispersión entre semillas.
4. Evaluación explícita de H2: correr el sistema con `uncertainty_stop_threshold` definido en varios valores de τ , y medir cuántas evaluaciones reales ahorra el criterio de parada respecto a un presupuesto fijo, sin pérdida de cobertura empírica.

6 Resultados

6.1 Configuración

Se evaluó el sistema sobre la función de Branin ($\mathcal{X} = [-5, 10] \times [0, 15]$, mínimo global conocido ≈ 0.397887), umbral de error objetivo 0.05. Aprendizaje activo: $n_{\text{init}} = 8$, presupuesto total 60, adquisición EI ($\xi = 0.01$), 2000 candidatos por iteración. A diferencia de un primer reporte preliminar de este trabajo (una única semilla), aquí reportamos: **15 semillas independientes**

para aprendizaje activo, **30 semillas** para el baseline aleatorio con presupuesto extendido a **2000 evaluaciones** (para eliminar la censura de un presupuesto fijo demasiado bajo), y un **barrido de τ** (5 semillas por valor) para evaluar H2 explícitamente.

6.2 H1: adquisición guiada vs. muestreo aleatorio

El aprendizaje activo alcanzó el umbral de error (≤ 0.05) en **22.1 ± 2.7** evaluaciones reales (media $\pm$ desvío, 15/15 semillas convergen; rango 18–27). El baseline aleatorio, con presupuesto extendido a 2000 evaluaciones, convergió en 27 de 30 semillas, con una **mediana de 762** evaluaciones (media 731.9, desvío 423.8 — alta varianza intrínseca del muestreo aleatorio en este benchmark); las 3 semillas restantes no alcanzaron el umbral ni con 2000 evaluaciones.

Esto reemplaza la estimación preliminar de un reporte anterior de este trabajo, que comparó 21 evaluaciones (1 semilla, aprendizaje activo) contra un baseline *censurado* en 60 evaluaciones (ninguna de 8 semillas convergió dentro de ese presupuesto), produciendo una reducción reportada de 65% caracterizada explícitamente en ese momento como un límite inferior conservador. Con datos de convergencia real y no censurados, la reducción medida es:

$$1 - \frac{22.1}{762} \approx \mathbf{97.1\%} \quad (12)$$

sobre la mediana del baseline, consistente con la dirección — y la magnitud conservadora — del resultado preliminar. La Figura 2 muestra la distribución completa de evaluaciones necesarias para cada método.

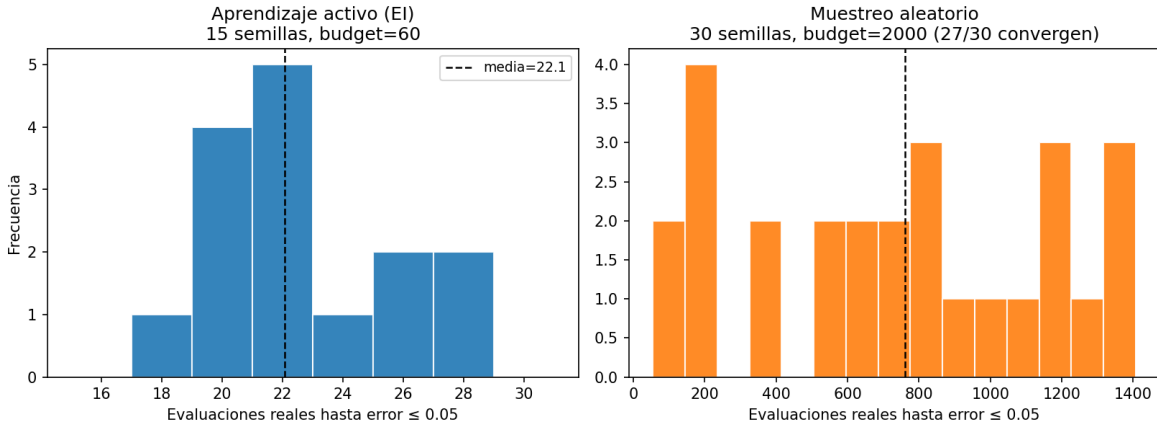


Figure 2: Distribución de evaluaciones reales necesarias para alcanzar error ≤ 0.05 . Izquierda: aprendizaje activo (EI), 15 semillas, muy concentrada. Derecha: muestreo aleatorio, 30 semillas con presupuesto extendido a 2000, ampliamente dispersa (nótese la diferencia de escala en el eje x).

6.3 Calibración de incertidumbre

Sobre $N = 300$ puntos de prueba muestreados uniformemente, la cobertura empírica a 2σ del GP final de aprendizaje activo (sin criterio de parada, control) fue **94.7–96.5%** según la semilla, consistentemente cercana al valor nominal esperado ($\approx 95.4\%$) bajo un GP bien calibrado — ver Tabla 1 para el detalle por valor de τ .

6.4 H2: criterio de parada por incertidumbre

Evaluamos el mecanismo de §4.3 con un barrido de $\tau \in \{15, 18, 20, 25\}$ (5 semillas cada uno), más un control sin criterio de parada. Un primer intento con valores de τ en la escala intuitiva del error del problema ($\tau \in \{0.05, 0.1\}$) **no disparó el criterio en ninguna semilla**: el $\max_{x \in C_t} \sigma_t(x)$ sobre 2000 candidatos uniformes se mantuvo consistentemente entre 18 y 22 durante todo el proceso, incluso en la última iteración. Esto ocurre porque, con un presupuesto de apenas 60 puntos sobre un dominio continuo 2D, siempre queda algún candidato en una región poco explorada del espacio, y esa región domina el máximo. La escala real de $\sigma_t(x)$ para candidatos no explorados está acotada por el prior del kernel (σ_f^2 hasta 10^3 , ver §4.1), no por la escala del error del problema.

τ	Evaluaciones reales (media)	Error final (media)	Cobertura 2σ (media)
15.0	49.8	0.0023	96.7%
18.0	41.4	0.0132	97.1%
20.0	37.2	0.0144	91.4%
25.0	28.6	0.0150	90.7%
sin umbral (control)	60.0	0.0011	96.5%

Table 1: H2: barrido de τ sobre la escala correcta del kernel (5 semillas por valor). El criterio de parada ahorra evaluaciones de forma creciente con τ , a costa de una cobertura decreciente.

Una vez recalibrado τ a la escala real del kernel, el mecanismo **sí ahorra evaluaciones reales** — hasta 52% con $\tau = 25$ respecto al control — pero con un trade-off claro frente a la calibración (Figura 3): la cobertura cae de 96.5% (control) a 90.7% ($\tau = 25$), por debajo del objetivo nominal de $\approx 95\%$. H2 queda evidenciada como mecanismo funcional, con dos salvedades importantes: (a) τ no es interpretable en la escala del problema y requiere calibrarse contra el kernel entrenado, lo cual es una limitación de usabilidad real del diseño actual (§7); y (b) el punto de τ que preserva cobertura nominal (≈ 15 – 18) ahorra sustancialmente menos que valores más agresivos.

7 Estudio de ablación propuesto

La Tabla 2 describe el diseño de ablación necesario para aislar qué componente del sistema es responsable de la reducción de costo observada. *Random*, *SmartSim (EI)* y *SmartSim (EI + τ)* ya fueron corridas con múltiples semillas (§6); GP + UCB y GP + PI quedan como comparación directa pendiente entre funciones de adquisición, ya que el código las soporta pero aún no se corrieron sistemáticamente.

Variante	GP	Incert.	Adquisición	τ	Estado
Random	—	—	—	—	corrido (30 sem., budget=2000)
GP + Random	✓	✓	aleatoria	no	pendiente
GP + UCB	✓	✓	UCB	no	código listo, no corrido
GP + PI	✓	✓	PI	no	código listo, no corrido
SmartSim (EI)	✓	✓	EI	no	corrido (15 semillas)
SmartSim (EI+ τ)	✓	✓	EI	sí	corrido (barrido τ , 5 sem./valor)

Table 2: Diseño de ablación. Permite separar el efecto de tener un GP con incertidumbre, el efecto de la función de adquisición específica, y el efecto del criterio de parada (τ). Actualizado tras el protocolo multi-semilla de §6.

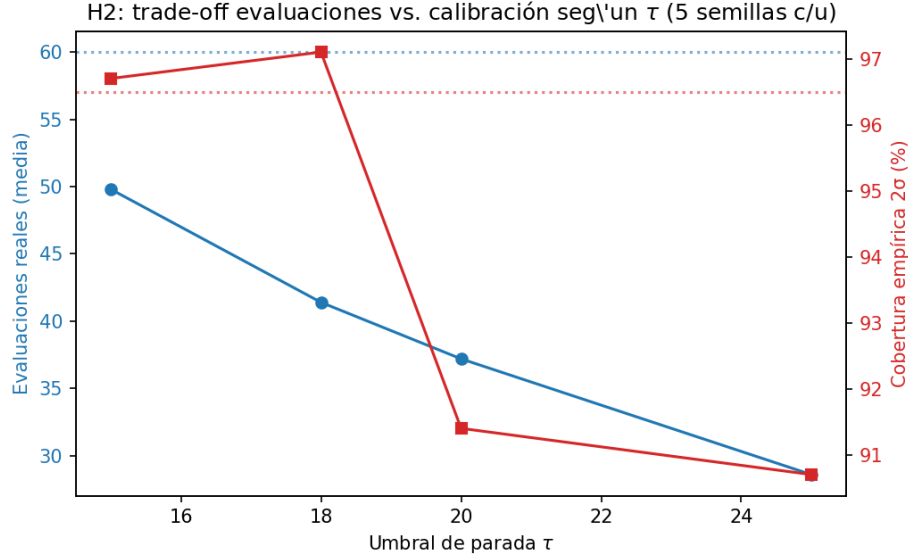


Figure 3: Trade-off entre evaluaciones reales ahorradas y cobertura empírica 2σ según el umbral τ . Líneas punteadas: valores de control (sin criterio de parada).

8 Limitaciones

1. **Reentrenamiento completo del GP.** El sustituto se reajusta desde cero en cada iteración activa sobre todo el historial acumulado; no hay actualización incremental. Esto tiene costo computacional creciente en n_t y no refleja necesariamente cómo se desplegaría el sistema en un escenario de streaming continuo. Este costo fue directamente observable en este trabajo: 15 semillas de aprendizaje activo tomaron ~ 3 minutos, mientras que 30 semillas de muestreo aleatorio con presupuesto extendido a 2000 evaluaciones tomaron segundos, precisamente porque el baseline no reentrena ningún modelo.
2. **τ no es interpretable en la escala del problema (hallazgo de §6.4).** El criterio de parada de H2 depende de la escala del prior del kernel, no del error del problema: valores intuitivos como $\tau \in \{0.05, 0.1\}$ nunca disparan el criterio, mientras que valores en la escala correcta ($\tau \approx 15\text{--}25$ en este benchmark) sí lo hacen, pero requieren conocer de antemano la escala típica de $\sigma_t(x)$ del kernel entrenado. Una versión práctica del sistema debería normalizar τ (p.ej. como fracción del prior σ_f) en vez de exponer un valor absoluto.
3. **Trade-off no resuelto entre ahorro y calibración en H2.** El barrido de §6.4 muestra que, a mayor τ , mayor ahorro de evaluaciones pero menor cobertura empírica (90.7% en el extremo evaluado, por debajo del 95% nominal). No hay aún un criterio automático para elegir τ que garantice una cobertura mínima objetivo.
4. **Un solo benchmark evaluado.** Branin es una función sintética 2D de baja dimensionalidad; Hartmann-6 está implementado pero no evaluado, y no hay aún un benchmark de un sistema dinámico o un simulador realista. Los hallazgos de §6 (incluyendo la escala de τ) son específicos a este benchmark y podrían no generalizar directamente a otras dimensionalidades o kernels.
5. **Optimización de la adquisición por búsqueda aleatoria.** Se usan 2000 candidatos uniformes en vez de optimización por gradiente sobre la función de adquisición, lo cual es razonable

en baja dimensión pero escala mal.

6. **Sin multi-fidelidad ni selección de lotes.** El sistema no combina un simulador barato con uno caro, ni selecciona múltiples puntos en paralelo (fuera de alcance, declarado en el repositorio original).

9 Trabajo futuro

- Ampliar el barrido de τ de H2 con más semillas por valor y un rango más fino, y diseñar un mecanismo de normalización de τ respecto al prior del kernel para que sea interpretable sin conocer de antemano la escala del problema.
- Correr las variantes de ablación pendientes (GP + UCB, GP + PI, GP + adquisición aleatoria) con el mismo protocolo multi-semilla usado para EI, para aislar el efecto específico de cada función de adquisición.
- Extender la evaluación a Hartmann-6 y a un sistema dinámico simple (masa-resorte-amortiguador o péndulo doble) como paso intermedio hacia simuladores realistas, verificando si los hallazgos de escala de τ se sostienen en mayor dimensionalidad.
- Explorar actualización incremental del GP para reducir el costo de reentrenamiento por iteración, motivado directamente por el costo computacional observado en el protocolo multi-semilla (§7).
- Investigar multi-fidelidad (combinar un simulador barato y uno caro) como extensión natural del criterio de decisión formalizado en §4.3.

10 Conclusión

Presentamos SmartSim, un sistema de aprendizaje activo que acopla un modelo sustituto probabilístico con un simulador de alta fidelidad. Formalizamos con precisión dos mecanismos distintos — la función de adquisición que decide dónde muestrear, y un criterio de parada basado en incertidumbre máxima que decide cuándo dejar de simular — y evaluamos ambos con un protocolo multi-semilla. H1 se confirma con una reducción real de evaluaciones cercana al 97% (mediana no censurada), muy por encima de una estimación preliminar previa del 65% construida sobre un baseline con presupuesto insuficiente. H2 también se confirma como mecanismo funcional, con un hallazgo de diseño relevante: el umbral τ solo es efectivo si se calibra en la escala del kernel entrenado, y existe un trade-off medible entre evaluaciones ahorradas y cobertura de incertidumbre preservada. Declaramos explícitamente las limitaciones que persisten — un solo benchmark, reentrenamiento completo del GP, ausencia de normalización de τ — como agenda concreta de trabajo futuro. Creemos que esta honestidad metodológica, sostenida ahora sobre evidencia multi-semilla en vez de corridas únicas, es la contribución más útil de este reporte para futuros trabajos sobre el mismo sistema.

Disponibilidad de código

El código completo, incluyendo el módulo del modelo sustituto, las funciones de adquisición, el orquestador y los tests, está disponible en: <https://github.com/rainvare/modelo-del-mundo>.

References

- [1] A. Marrel and B. Iooss. *Probabilistic surrogate modeling by Gaussian process: A review on recent insights in estimation and validation*. Reliability Engineering & System Safety, 247:110094, 2024.
- [2] D. H. Svendsen, L. Martino, and G. Camps-Valls. *Active emulation of computer codes with Gaussian processes – Application to remote sensing*. arXiv:1912.06552, 2019.
- [3] M. Moustapha, S. Marelli, and B. Sudret. *Active learning for structural reliability: survey, general framework and benchmark*. Structural Safety, 96:102174, 2022. arXiv:2106.01713.
- [4] D. R. Jones, M. Schonlau, and W. J. Welch. *Efficient global optimization of expensive black-box functions*. Journal of Global Optimization, 13(4):455–492, 1998.
- [5] J. Snoek, H. Larochelle, and R. P. Adams. *Practical Bayesian optimization of machine learning algorithms*. Advances in Neural Information Processing Systems, 25, 2012.